

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from scipy.stats import randint
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
In [2]: df=pd.read_csv ("loan-data.csv")
df.info()
df.isna().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               367 non-null   object
1   Gender                356 non-null   object
2   Married               367 non-null   object
3   Dependents            357 non-null   object
4   Education              367 non-null   object
5   Self_Employed         344 non-null   object
6   ApplicantIncome        367 non-null   int64
7   CoapplicantIncome      367 non-null   int64
8   LoanAmount             362 non-null   float64
9   Loan_Amount_Term       361 non-null   float64
10  Credit_History         338 non-null   float64
11  Property_Area          367 non-null   object
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
```

```
Out[2]: Loan_ID      0
Gender      11
Married     0
Dependents  10
Education   0
Self_Employed  23
ApplicantIncome  0
CoapplicantIncome  0
LoanAmount    5
Loan_Amount_Term  6
Credit_History  29
Property_Area  0
dtype: int64
```

```
In [3]: df.describe()
```

Out[3]:

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_Hist
count	367.000000	367.000000	362.000000	361.000000	338.000000
mean	4805.599455	1569.577657	136.132597	342.537396	0.8254
std	4910.685399	2334.232099	61.366652	65.156643	0.3801
min	0.000000	0.000000	28.000000	6.000000	0.0000
25%	2864.000000	0.000000	100.250000	360.000000	1.0000
50%	3786.000000	1025.000000	125.000000	360.000000	1.0000
75%	5060.000000	2430.500000	158.000000	360.000000	1.0000
max	72529.000000	24000.000000	550.000000	480.000000	1.0000

In [4]: `df["Credit_History"].value_counts()`

Out[4]: Credit_History
 1.0 279
 0.0 59
 Name: count, dtype: int64

In [5]: `min_value=df["LoanAmount"].min()
 max_value=df["LoanAmount"].max()
 print("Minimum:",min_value)
 print("Maximum:",max_value)`

Minimum: 28.0
 Maximum: 550.0

In [6]: `#iqr method
 Q1=df["LoanAmount"].quantile(0.25)
 Q3=df["LoanAmount"].quantile(0.75)
 IQR=Q3-Q1

 outliers=df[(df["LoanAmount"]<(Q1-1.5*IQR))|(df["LoanAmount"]>(Q3+1.5*IQR))]
 print("Outliers in LoanAmount column:\n",outliers)`

Outliers in LoanAmount column:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
8	LP001059	Male	Yes	2	Graduate	NaN	
18	LP001108	Male	Yes	0	Graduate	No	
24	LP001149	Male	Yes	0	Graduate	No	
27	LP001169	Male	Yes	0	Graduate	No	
81	LP001428	Male	Yes	3+	Graduate	No	
83	LP001446	Male	Yes	0	Graduate	No	
91	LP001483	Male	Yes	3+	Graduate	No	
96	LP001500	Male	Yes	1	Graduate	No	
124	LP001655	Female	No	0	Graduate	No	
143	LP001791	Male	Yes	0	Graduate	Yes	
144	LP001794	Male	Yes	2	Graduate	Yes	
189	LP002059	Male	Yes	2	Graduate	No	
194	LP002077	Male	Yes	1	Graduate	No	
284	LP002570	Female	Yes	2	Graduate	No	
285	LP002572	Male	Yes	1	Graduate	NaN	
331	LP002825	Male	Yes	3+	Graduate	No	
345	LP002878	Male	Yes	3+	Graduate	No	
350	LP002899	Male	Yes	2	Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
8	13633	0	280.0	240.0	
18	9226	7916	300.0	360.0	
24	5400	4380	290.0	360.0	
27	7500	3750	275.0	360.0	
81	72529	0	360.0	360.0	
83	8449	0	257.0	360.0	
91	13518	0	390.0	360.0	
96	3333	4200	256.0	360.0	
124	12500	0	300.0	360.0	
143	32000	0	550.0	360.0	
144	10890	0	260.0	12.0	
189	7874	3967	336.0	360.0	
194	10000	2690	412.0	360.0	
284	10000	11666	460.0	360.0	
285	8750	0	297.0	360.0	
331	9699	0	300.0	360.0	
345	8334	0	260.0	360.0	
350	8667	0	254.0	360.0	

	Credit_History	Property_Area
8	1.0	Urban
18	1.0	Urban
24	1.0	Urban
27	1.0	Urban
81	1.0	Urban
83	1.0	Rural
91	1.0	Rural
96	1.0	Urban
124	0.0	Urban
143	NaN	Semiurban
144	1.0	Rural
189	1.0	Rural
194	1.0	Semiurban
284	1.0	Urban

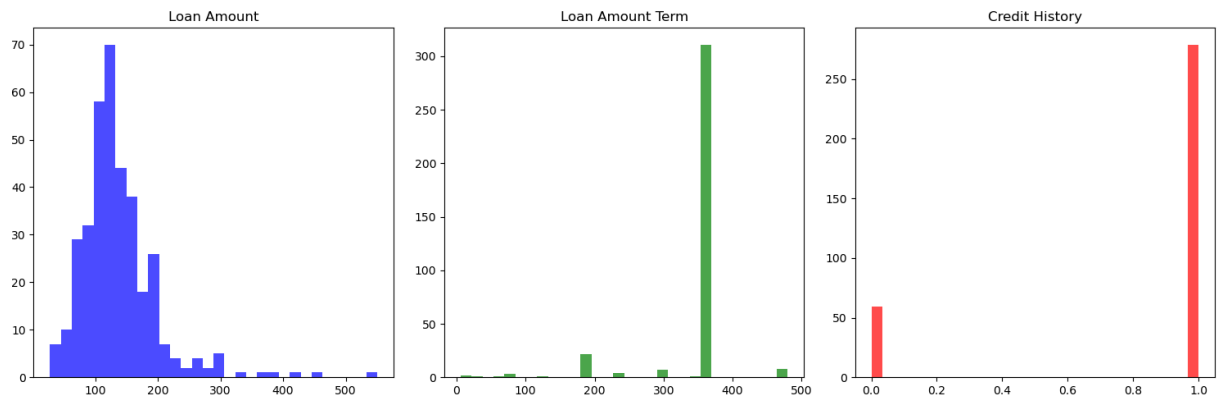
285	1.0	Urban
331	1.0	Urban
345	1.0	Urban
350	1.0	Rural

```
In [7]: # distribution of numerical columns with missing values
# create a 1x3 grid of subplots
fig, axes=plt.subplots(1,3, figsize=(15,5))
#plot histogram
axes[0].hist(df.LoanAmount, bins=30, color="blue", alpha=0.7)
axes[0].set_title("Loan Amount")

axes[1].hist(df.Loan_Amount_Term, bins=30, color="green", alpha=0.7)
axes[1].set_title("Loan Amount Term")

axes[2].hist(df.Credit_History, bins=30, color="red", alpha=0.7)
axes[2].set_title("Credit History")

#Adjust layout for better visualization
plt.tight_layout()
plt.show()
```



```
In [8]: # replace missing value with numerical feature with median and mode:
df["LoanAmount"] =df["LoanAmount"].fillna(df["LoanAmount"].median())
df["Loan_Amount_Term"] =df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"] .mode(
df["Credit_History"] =df["Credit_History"].fillna(df["Credit_History"].mode()[0])

# Replace missing value with categorical feature with mode:
df["Gender"] =df["Gender"].fillna(df["Gender"].mode()[0])
df["Dependents"] =df["Dependents"].fillna(df["Dependents"].mode()[0])
df["Self_Employed"] =df["Self_Employed"].fillna(df["Self_Employed"].mode()[0])

df.isna().sum()
df.head()
```

Out[8]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001015	Male	Yes	0	Graduate	No	5720
1	LP001022	Male	Yes	1	Graduate	No	3076
2	LP001031	Male	Yes	2	Graduate	No	5000
3	LP001035	Male	Yes	2	Graduate	No	2340
4	LP001051	Male	No	0	Not Graduate	No	3276

In [9]: *# Converting loan_amount_term and credit_history to integer*
`df["Loan_Amount_Term"]=df["Loan_Amount_Term"].astype("int64")`
`df["Credit_History"]=df["Credit_History"].astype("int64")`
`df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               367 non-null   object
1   Gender                367 non-null   object
2   Married               367 non-null   object
3   Dependents            367 non-null   object
4   Education             367 non-null   object
5   Self_Employed         367 non-null   object
6   ApplicantIncome       367 non-null   int64
7   CoapplicantIncome     367 non-null   int64
8   LoanAmount            367 non-null   float64
9   Loan_Amount_Term      367 non-null   int64
10  Credit_History        367 non-null   int64
11  Property_Area         367 non-null   object
dtypes: float64(1), int64(4), object(7)
memory usage: 34.5+ KB
```

In [10]: *#EXPLORATRY DATA ANALYSIS*
#Set style for beter visualization
`sns.set_style("whitegrid")`

#plot boxplot for numerical variable
`plt.figure(figsize=(15,5))`

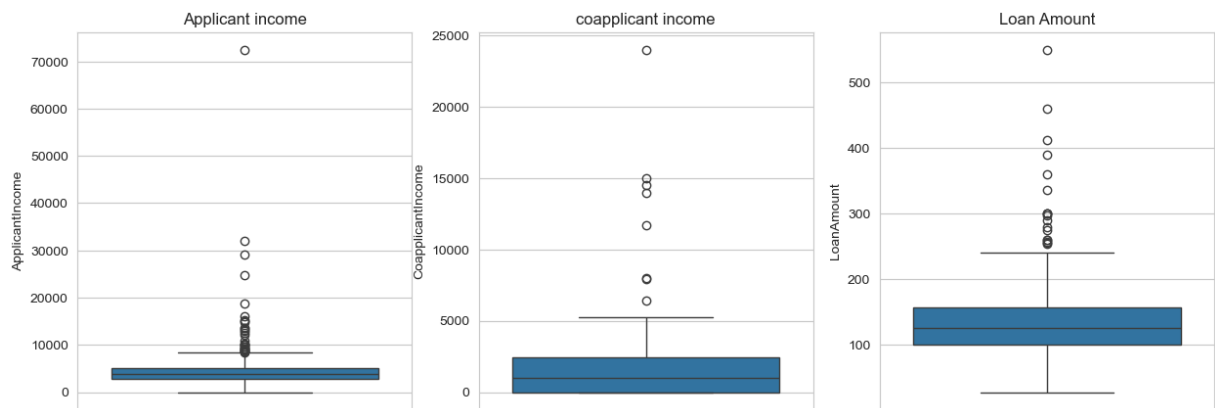
`plt.subplot(1,3,1)`
`sns.boxplot(y=df["ApplicantIncome"])`
`plt.title("Applicant income")`

`plt.subplot(1,3,2)`
`sns.boxplot(y=df["CoapplicantIncome"])`
`plt.title("coapplicant income")`

`plt.subplot(1,3,3)`
`sns.boxplot(y=df["LoanAmount"])`

```
plt.title("Loan Amount")
```

```
plt.show()
```



```
In [11]: #EXPLORATRY DATA ANALYSIS
#Set style for beter visualization
sns.set_style("whitegrid")

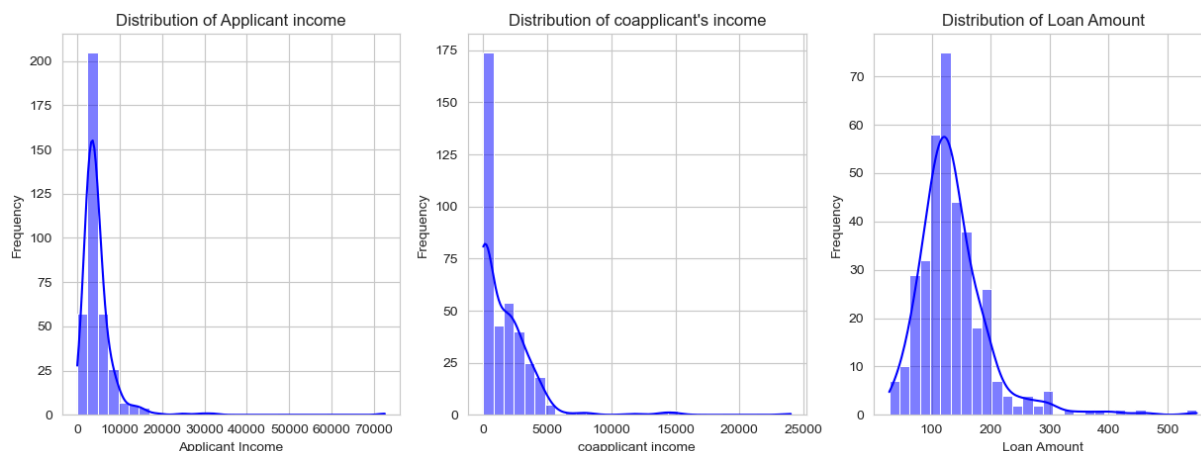
#plot boxplot for numerical variable
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
sns.histplot(df["ApplicantIncome"], bins=30, kde=True, color="blue")
plt.title("Distribution of Applicant income")
plt.xlabel("Applicant Income")
plt.ylabel("Frequency")

plt.subplot(1,3,2)
sns.histplot(df["CoapplicantIncome"], bins=30, kde=True, color="blue")
plt.title("Distribution of coapplicant's income")
plt.xlabel("coapplicant income")
plt.ylabel("Frequency")

plt.subplot(1,3,3)
sns.histplot(df["LoanAmount"], bins=30, kde=True, color="blue")
plt.title("Distribution of Loan Amount")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")

plt.show()
```



```
In [12]: #apply log tranformation to loanAmount, ApplicationIncome and CoapplicationIncome
df["LoanAmount_log"] = np.log1p(df["LoanAmount"])
df["ApplicantIncome_log"] = np.log1p(df["ApplicantIncome"])
df["CoapplicantIncome_log"] = np.log1p(df["CoapplicantIncome"])
```

```
In [13]: #checking distribution again
sns.set_style("whitegrid")

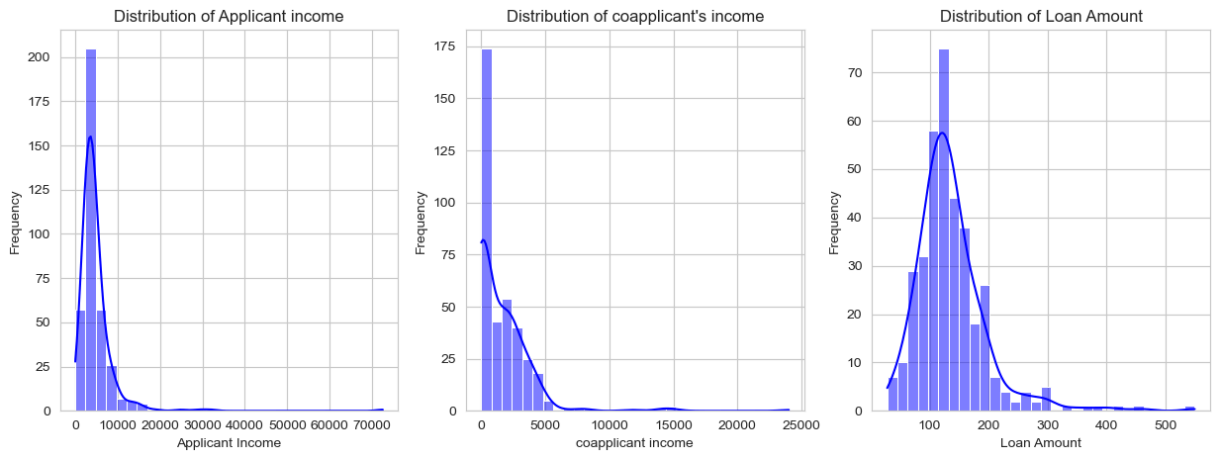
#plot boxplot for numerical variable
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
sns.histplot(df["ApplicantIncome"], bins=30, kde=True, color="blue")
plt.title("Distribution of Applicant income")
plt.xlabel("Applicant Income")
plt.ylabel("Frequency")

plt.subplot(1,3,2)
sns.histplot(df["CoapplicantIncome"], bins=30, kde=True, color="blue")
plt.title("Distribution of coapplicant's income")
plt.xlabel("coapplicant income")
plt.ylabel("Frequency")

plt.subplot(1,3,3)
sns.histplot(df["LoanAmount"], bins=30, kde=True, color="blue")
plt.title("Distribution of Loan Amount")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")

plt.show()
```



```
In [14]: df.head(10)
```

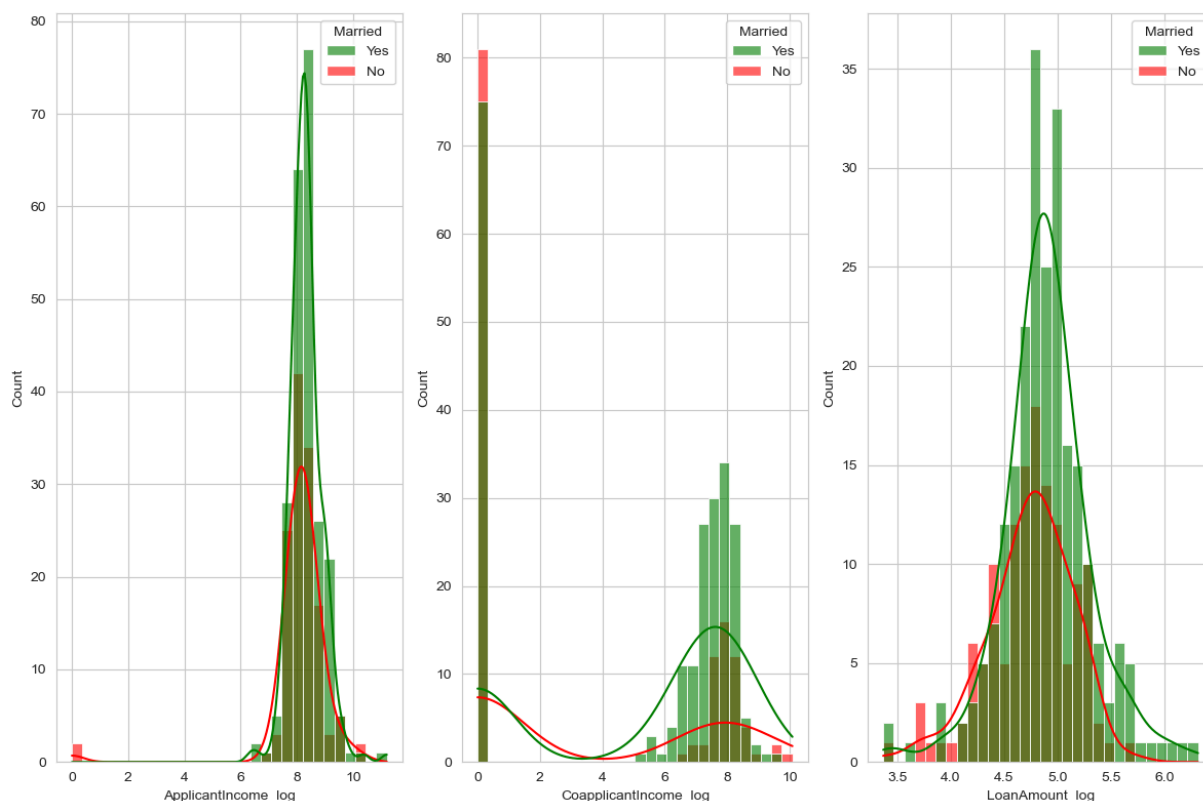
Out[14]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Credit_History
0	LP001015	Male	Yes	0	Graduate	No	5720	1
1	LP001022	Male	Yes	1	Graduate	No	3076	1
2	LP001031	Male	Yes	2	Graduate	No	5000	1
3	LP001035	Male	Yes	2	Graduate	No	2340	1
4	LP001051	Male	No	0	Not Graduate	No	3276	1
5	LP001054	Male	Yes	0	Not Graduate	Yes	2165	1
6	LP001055	Female	No	1	Not Graduate	No	2226	1
7	LP001056	Male	Yes	2	Not Graduate	No	3881	1
8	LP001059	Male	Yes	2	Graduate	No	13633	1
9	LP001067	Male	No	0	Not Graduate	No	2400	1

```
In [15]: #Distribution of numerical columns by loan status
sns.set_style("whitegrid")
# define numerical column
numerical_cols=["ApplicantIncome_log","CoapplicantIncome_log","LoanAmount_log"]
#define custom color for Married (Y=yes, N=no)
palette = {"Yes": "green", "No": "red"}
#create histogram
plt.figure(figsize=(12,8))
for i,col in enumerate(numerical_cols,1):
    plt.subplot(1,3,i)
    sns.histplot(data=df, x=col, hue="Married", kde=True, bins=30, alpha=0.6, palet
```



```
plt.tight_layout()
plt.show()
```



```
In [16]: print(df[categorical_cols].info())
for col in categorical_cols:
    print(col,df[col].unique())
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[16], line 1
----> 1 print(df[categorical_cols].info())
      2 for col in categorical_cols:
      3     print(col,df[col].unique())

NameError: name 'categorical_cols' is not defined
```

```
In [18]: # Distribution of categorical columns
#set style for plot
sns.set_style("whitegrid")

#define custom color for married (Y=Green, N=Red)
custom_palette = {"Graduate": "green","Not Graduate": "red"}

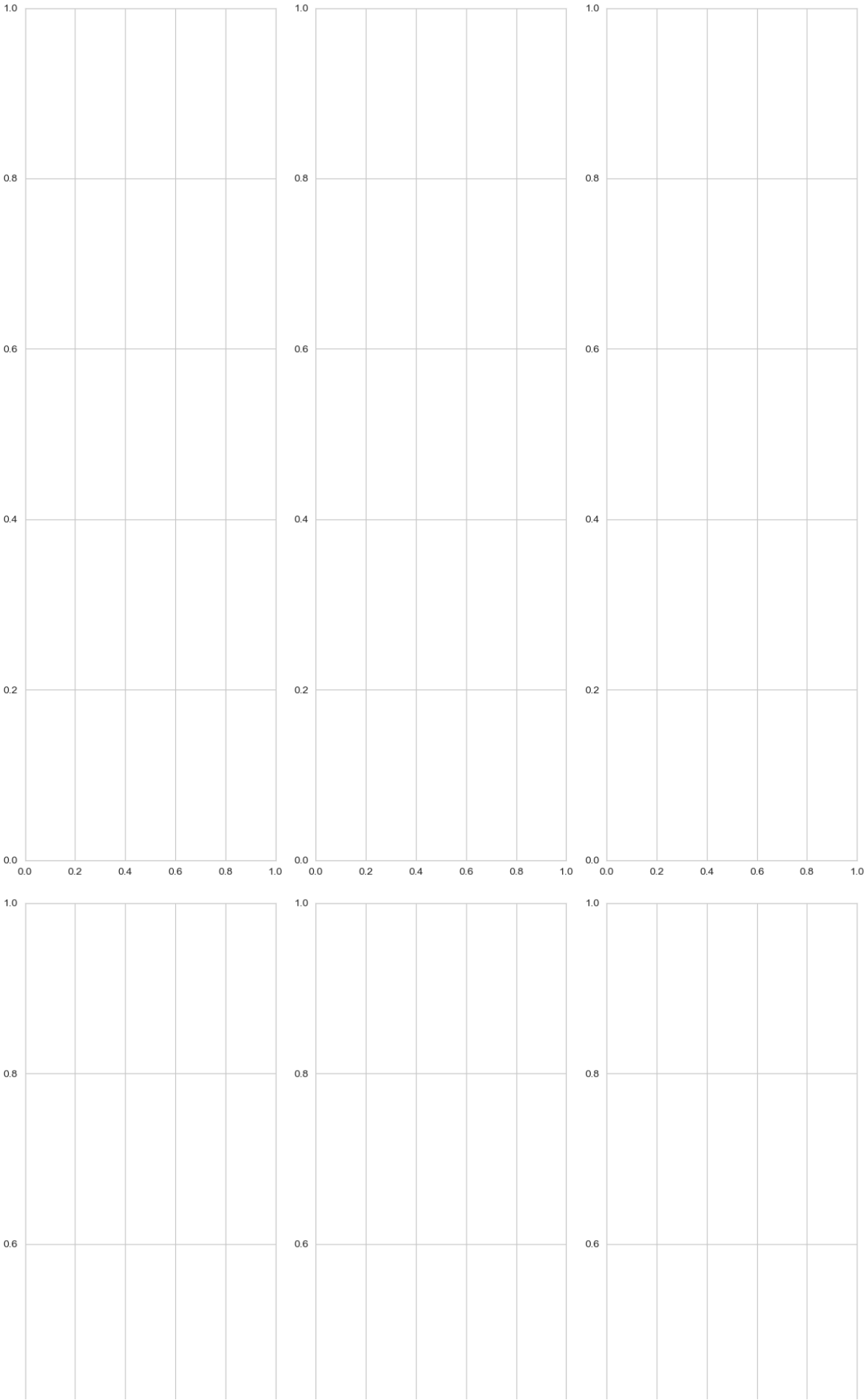
#list of categorical columns
categorical_cols=["Married","Dependents","Gender","Self_Employed","Credit_History",
import math
n_cols=3
n_rows=math.ceil(len(categorical_cols)/n_cols)

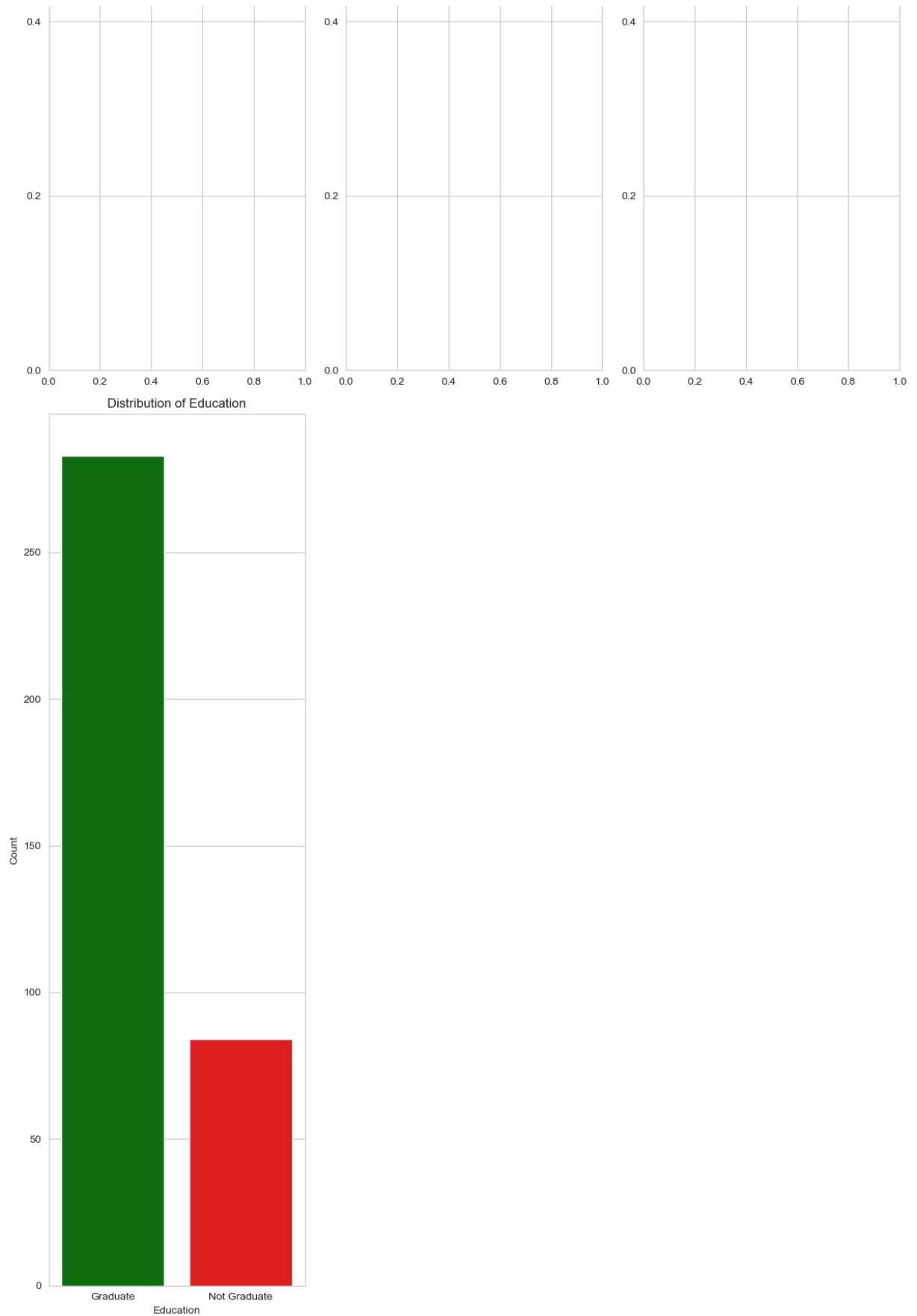
# create count plot for all categorical variable
plt.figure(figsize=(12,12*n_rows))
```

```
for i, col in enumerate(categorical_cols, 1):
    plt.subplot(n_rows, n_cols, i) #Arrange in 4x2 grid

#Apply custom palette to only married, other use default color
    if col == "Education":
        sns.countplot(x=df["Education"], data=df, hue="Education", palette=custom_palet
    else:
        sns.countplot(x=df["Education"], data=df, color="blue") #use a default color
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Count")

plt.tight_layout()
plt.show()
```





```
In [19]: # set style for plot
sns.set_style("whitegrid")
```

```
#Define custom color for loan status (y=Green, N=Red)
cutom_palette = {"Female":"green", "Male":"Red"}

#List of categories columns to visualize (excluding loan_status itself)
categorical_cols=["Married","Dependents","Education","Self_Employed","Credit_Histor

#creat count plot for all categorical variable against Loan_status
plt.figure(figsize=(12,12))
for i, col in enumerate(categorical_cols,1):
    plt.subplot(4,2,i) #arrange 4x2 grid
    sns.countplot(x=col, hue="Gender", data=df, palette=custom_palette)
    plt.title(f" Gender by{col}")
    plt.xlabel(col)
    plt.ylabel("count")
plt.tight_layout()
plt.show()
```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[19], line 14
    12 for i, col in enumerate(categorical_cols,1):
    13     plt.subplot(4,2,i) #arrange 4x2 grid
--> 14     sns.countplot(x=col, hue="Gender", data=df, palette=custom_palette)
    15     plt.title(f" Gender by{col}")
    16     plt.xlabel(col)

File C:\anacondafile\Lib\site-packages\seaborn\categorical.py:2660, in countplot(data, x, y, hue, order, hue_order, orient, color, palette, saturation, fill, hue_norm, stat, width, dodge, gap, log_scale, native_scale, formatter, legend, ax, **kwargs)
    2657 palette, hue_order = p._hue_backcompat(color, palette, hue_order)
    2659 saturation = saturation if fill else 1
-> 2660 p.map_hue(palette=palette, order=hue_order, norm=hue_norm, saturation=saturation)
    2661 color = _default_color(ax.bar, hue, color, kwargs, saturation)
    2663 count_axis = {"x": "y", "y": "x"}[p.orient]

File C:\anacondafile\Lib\site-packages\seaborn\_base.py:838, in VectorPlotter.map_hue(self, palette, order, norm, saturation)
    837 def map_hue(self, palette=None, order=None, norm=None, saturation=1):
--> 838     mapping = HueMapping(self, palette, order, norm, saturation)
    839     self._hue_map = mapping

File C:\anacondafile\Lib\site-packages\seaborn\_base.py:150, in HueMapping.__init__(self, plotter, palette, order, norm, saturation)
    147 elif map_type == "categorical":
    149     cmap = norm = None
--> 150     levels, lookup_table = self.categorical_mapping(
    151         data, palette, order,
    152     )
    154 # --- Option 3: datetime mapping
    155
    156 else:
    157     # TODO this needs actual implementation
    158     cmap = norm = None

File C:\anacondafile\Lib\site-packages\seaborn\_base.py:234, in HueMapping.categorical_mapping(self, data, palette, order)
    232 if any(missing):
    233     err = "The palette dictionary is missing keys: {}"
--> 234     raise ValueError(err.format(missing))
    236 lookup_table = palette
    238 else:

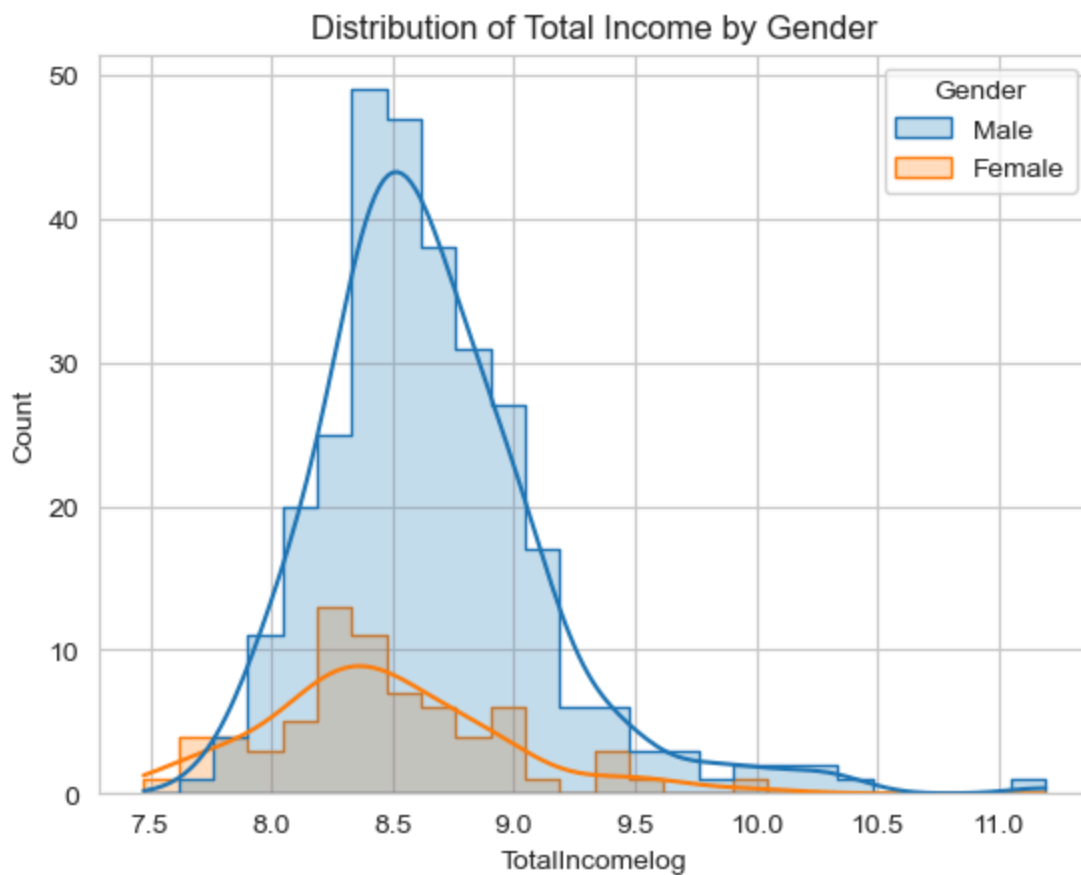
ValueError: The palette dictionary is missing keys: {'Male', 'Female'}

```



```
In [20]: # Feature engineering
#create a total income column
df["TotalIncome"] =df["ApplicantIncome"]+df["CoapplicantIncome"]
df["TotalIncomelog"]=np.log1p(df["TotalIncome"])

sns.histplot(data=df, x="TotalIncomelog", hue="Gender", kde=True,element="step")
plt.title("Distribution of Total Income by Gender")
plt.show()
```



```
In [21]: df.info()
#Dropping columns that are not necessary
cols_to_drop=["ApplicantIncome","CoapplicantIncome","ApplicantIncome_log","Coapplic
```

```
df=df.drop(columns=cols_to_drop,axis=1)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Loan_ID                               367 non-null    object
1   Gender                                367 non-null    object
2   Married                               367 non-null    object
3   Dependents                            367 non-null    object
4   Education                             367 non-null    object
5   Self_Employed                         367 non-null    object
6   ApplicantIncome                       367 non-null    int64
7   CoapplicantIncome                     367 non-null    int64
8   LoanAmount                            367 non-null    float64
9   Loan_Amount_Term                      367 non-null    int64
10  Credit_History                        367 non-null    int64
11  Property_Area                         367 non-null    object
12  LoanAmount_log                        367 non-null    float64
13  ApplicantIncome_log                  367 non-null    float64
14  CoapplicantIncome_log                 367 non-null    float64
15  TotalIncome                          367 non-null    int64
16  TotalIncomelog                       367 non-null    float64
dtypes: float64(5), int64(5), object(7)
memory usage: 48.9+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Gender                                367 non-null    object
1   Married                               367 non-null    object
2   Dependents                            367 non-null    object
3   Education                             367 non-null    object
4   Self_Employed                         367 non-null    object
5   LoanAmount                            367 non-null    float64
6   Loan_Amount_Term                      367 non-null    int64
7   Credit_History                        367 non-null    int64
8   Property_Area                         367 non-null    object
9   LoanAmount_log                        367 non-null    float64
10  TotalIncome                          367 non-null    int64
11  TotalIncomelog                       367 non-null    float64
dtypes: float64(3), int64(3), object(6)
memory usage: 34.5+ KB
```

```
In [22]: #ENCODING CATEGORICAL VARIABLES
Categorical_features=df.select_dtypes(include=["object"]).columns.tolist()
Categorical_features
```



```
Out[22]: ['Gender',
          'Married',
          'Dependents',
          'Education',
          'Self_Employed',
          'Property_Area']
```

```
In [23]: #use pd.get_dummies for categorical variables
df_encoded=pd.get_dummies(df,columns=Categorical_features,drop_first=True)
#Ensure all columns are numeric (0s and 1s)
df_encoded=df_encoded.astype(int)
print(df_encoded.head())
```

	LoanAmount	Loan_Amount_Term	Credit_History	LoanAmount_log	TotalIncome	\
0	110	360	1	4	5720	
1	126	360	1	4	4576	
2	208	360	1	5	6800	
3	100	360	1	4	4886	
4	78	360	1	4	3276	

	TotalIncomeLog	Gender_Male	Married_Yes	Dependents_1	Dependents_2	\
0	8	1	1	0	0	
1	8	1	1	1	0	
2	8	1	1	0	1	
3	8	1	1	0	1	
4	8	1	0	0	0	

	Dependents_3+	Education_Not Graduate	Self_Employed_Yes	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	1	0	

	Property_Area_Semiurban	Property_Area_Urban
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1

```
In [24]: df_encoded.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   LoanAmount                           367 non-null    int64
1   Loan_Amount_Term                     367 non-null    int64
2   Credit_History                       367 non-null    int64
3   LoanAmount_log                       367 non-null    int64
4   TotalIncome                         367 non-null    int64
5   TotalIncome_log                     367 non-null    int64
6   Gender_Male                         367 non-null    int64
7   Married_Yes                         367 non-null    int64
8   Dependents_1                       367 non-null    int64
9   Dependents_2                       367 non-null    int64
10  Dependents_3+                       367 non-null    int64
11  Education_Not Graduate              367 non-null    int64
12  Self_Employed_Yes                  367 non-null    int64
13  Property_Area_Semiurban             367 non-null    int64
14  Property_Area_Urban                 367 non-null    int64
dtypes: int64(15)
memory usage: 43.1 KB
```

```
In [25]: # feature scaling
# ApplicantIncome, LoanAmount, and coapplicantIncome have a very different scale
# standardization helps the model learn better
scaler = StandardScaler()
numerical_features = ["LoanAmount", "Loan_Amount_Term", "TotalIncome"]
df[numerical_features] = scaler.fit_transform(df[numerical_features])
```

```
In [26]: x = df_encoded.drop("Married_Yes", axis=1)
y = df_encoded["Married_Yes"]
x.head()
```

```
Out[26]:
```

	LoanAmount	Loan_Amount_Term	Credit_History	LoanAmount_log	TotalIncome	TotalIncome_log
0	110	360	1	4	5720	3.85
1	126	360	1	4	4576	3.66
2	208	360	1	5	6800	3.93
3	100	360	1	4	4886	3.69
4	78	360	1	4	3276	3.52



```
In [27]: y.head()
```

```
Out[27]: 0    1
1    1
2    1
3    1
4    0
Name: Married_Yes, dtype: int64
```

```
In [28]: #Feature Scaling
#LoanAmount, CoapplicantIncome, ApplicantIncome have a very different scales
#Standardization helps the model learn better
Scaler=StandardScaler()
numerical_features=["LoanAmount", "Loan_Amount_Term", "TotalIncome"]
df[numerical_features]=Scaler.fit_transform(df[numerical_features])

In [29]: #splitting data base
#split data into training and testing set(80% train,20% test)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42,st

In [30]: #modeling
def model_eval(model,x,y):
    #Train the model
    model.fit(x_train,y_train)
    #predict on the test data
    y_pred=model.predict(x_test)
    #Accuracy score
    accuracy=accuracy_score(y_test,y_pred)
    print(f"{model}\n \nAccuracy:{accuracy}")
    print()

    #cross validation
    cv_score=cross_val_score(model,x,y,cv=10,scoring="accuracy")
    print(f"Average cross-validation score: {np.mean(cv_score)}")
    print()

    #confusion marix
    print("Confusion Matrix:")
    print(confusion_matrix(y_test,y_pred))
    print()

    #classification report
    print("Classification Report:")
    print(classification_report(y_test,y_pred))

In [31]: #Lodgistic regression
lr_model = LogisticRegression(max_iter=10000,class_weight="balanced")
model_eval(lr_model,x,y)
```

```
LogisticRegression(class_weight='balanced', max_iter=10000)
```

Accuracy:0.7702702702702703

Average cross-validation score: 0.6650150150150151

Confusion Matrix:

```
[[23  4]
 [13 34]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.64	0.85	0.73	27
1	0.89	0.72	0.80	47
accuracy			0.77	74
macro avg	0.77	0.79	0.77	74
weighted avg	0.80	0.77	0.77	74

```
In [32]: #Random forest classiffier
rf_model=RandomForestClassifier(class_weight="balanced")
model_eval(rf_model,x,y)
```

RandomForestClassifier(class_weight='balanced')

Accuracy:0.7297297297297297

Average cross-validation score: 0.6506006006006005

Confusion Matrix:

```
[[17 10]
 [10 37]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.63	0.63	0.63	27
1	0.79	0.79	0.79	47
accuracy			0.73	74
macro avg	0.71	0.71	0.71	74
weighted avg	0.73	0.73	0.73	74

```
In [33]: #Support Vector Machine
sv_model = SVC(class_weight="balanced")
model_eval(sv_model,x,y)
```

```
SVC(class_weight='balanced')
```

```
Accuracy:0.5405405405405406
```

```
Average cross-validation score: 0.5688438438438438
```

```
Confusion Matrix:
```

```
[[18  9]
 [25 22]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.42	0.67	0.51	27
1	0.71	0.47	0.56	47
accuracy			0.54	74
macro avg	0.56	0.57	0.54	74
weighted avg	0.60	0.54	0.55	74

```
In [34]: #Decision Tree classifier
dt_model = DecisionTreeClassifier(class_weight="balanced")
model_eval(dt_model,x,y)
```

```
DecisionTreeClassifier(class_weight='balanced')
```

```
Accuracy:0.6486486486486487
```

```
Average cross-validation score: 0.6343843843843844
```

```
Confusion Matrix:
```

```
[[13 14]
 [12 35]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.52	0.48	0.50	27
1	0.71	0.74	0.73	47
accuracy			0.65	74
macro avg	0.62	0.61	0.61	74
weighted avg	0.64	0.65	0.65	74